全栈开发技术期末项目报告

阅记

阅读与书摘管理应用

修订版 v2 / 配套源码 v5

项目名称	阅记阅读与书摘管理应用
姓名	周宇斌
学号	20246085
班级	软件2401

2026 年 10 月 21 日

依据实际源码、Linux历史记录及Windows本机验收修订。当前代码93项自动化测试、71项独立架构断言通过；本报告配套源码v5，应用与测试源码同已验收的v4，仅README更新。

目录

项目背景与意义 2

1 需求分析 2

1.1 功能需求 2

1.2 非功能需求 3

2 系统设计 3

2.1 架构设计 3

2.2 功能模块设计 4

2.3 数据/数据库设计 4

2.4 接口与类设计 4

2.5 技术选型说明 5

3 系统实现 5

3.1 关键模块的核心实现 5

3.2 核心功能与UI截图 7

3.3 难点和解决方案 14

4 系统测试 15

4.1 测试环境 15

4.2 测试用例 16

4.3 测试截图 17

5 总结与展望 20

项目背景与意义

个人阅读中，书籍资料、阅读活动与摘录通常分散记录。阅记以本地离线应用统一管理这三类信息，并由阅读记录推导当前进度、计划状态和统计结果，使用户能够追踪阅读活动、整理感想及恢复自己的数据。

项目不提供电子书阅读器、账号登录、联网检索或云同步。其业务边界是“记录阅读”，用户录入的阅读记录表示自身活动声明，不能据此推断所有未记录页段均已读过。演示数据中的书名、作者、摘录和感想均明确标注为虚构资料。

1 需求分析

1.1 功能需求

应用实现录入书籍、制定计划、记录阅读、整理书摘和查看统计的完整闭环。十个独立业务页面及主要能力如下。

页面	已实现能力
阅读概览	在读书籍、近期记录、待完成计划、全量累计指标、书库与详情入口。
我的书库	书名或作者搜索、未开始/在读/已完成筛选、每页10本分页、新增及详情入口。
书籍详情	资料、当前进度、累计活动、记录增改删弹窗、关联书摘与计划、级联删除确认。
书籍编辑	书名、作者、分类、总页数与简介编辑，校验已有最大引用页码。
书摘列表	正文或书名搜索、收藏筛选、编辑、收藏切换和删除。
书摘编辑	选择书籍、编辑摘录与感想、可选页码和收藏状态。
阅读计划	查看进行中、已完成与已逾期计划，编辑和删除。
计划编辑	选择书籍、设定目标日期、根据剩余页数和天数展示建议。
阅读统计	累计页数、分钟数、完成书籍数与近七日阅读趋势。
设置与数据	浅深主题、空库载入虚构演示数据、JSON导出与带自动备份的恢复。

初始化页、主窗口、主容器和阅读记录弹窗不用于补足十个业务页面。编辑未保存时，返回、切换菜单与关闭窗口共用保存、放弃、取消离开语义；保存失败保留草稿。列表返回刷新数据，保留当前搜索条件。

1.2 非功能需求

业务数据保存在用户本地应用数据目录，断网后核心业务可用。首次依赖还原需要NuGet来源；“运行离线”不等于“首次还原离线”。数据库写入串行，跨表修改采用事务。异步命令处理IO、忙碌状态和错误重试。

界面使用中文标签、侧边导航、统一表单及浅深主题。原生文件选择器使用系统语言，Linux历史测试中的GTK选择器部分文字为英文；Windows本机使用实际系统文件选择器。代码采用UI与Library职责分离，业务服务通过接口注入，便于替换依赖测试。本文不提供未经测量的性能指标，也不宣称已完成大规模数据压力测试。

2 系统设计

2.1 架构设计

解决方案包含ReadJournal、ReadJournal.Library及ReadJournal.UnitTest三个工程。UI负责视图、应用资源、定位器、导航实现、文件选择和提示；Library负责模型、规则、存储、统计、备份及ViewModel，不引用Avalonia控件；测试工程使用xUnit、Moq和临时真实SQLite数据库。

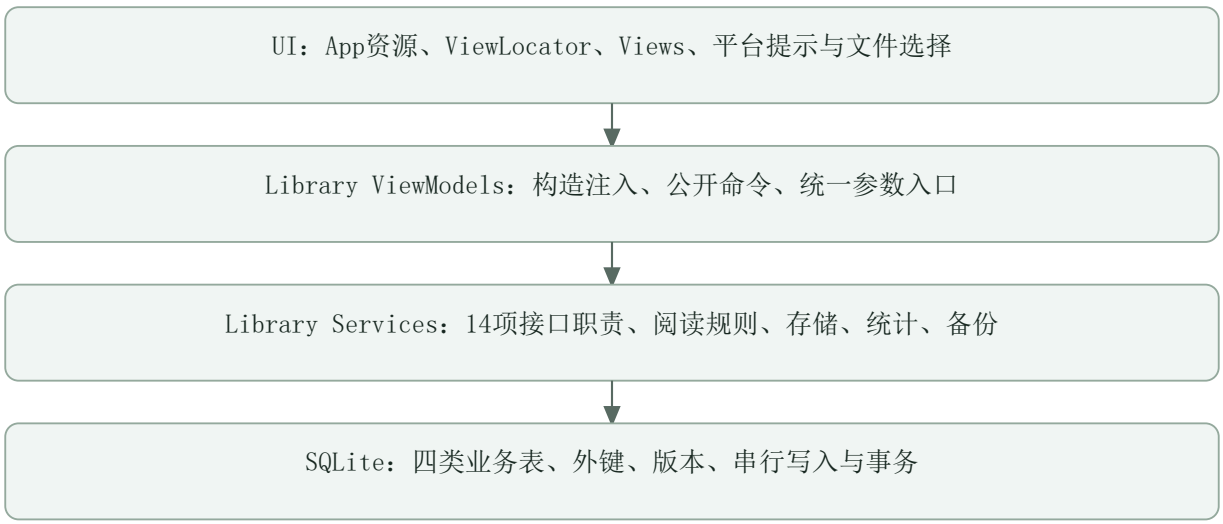


图 1 依据实际工程组织绘制的架构图

保留DailyPoetryA的完整组装机制：ServiceLocator构造内局部ServiceCollection注册、BuildServiceProvider构建、只读IServiceProvider保存、属性GetService解析；Current通过Application资源读取并缓存。全部服务和ViewModel为单例，构造函数只保存依赖并建立命令，不启动导航或加载。

根导航替换MainWindowViewModel.Content；菜单导航通过SetMenuAndContent清空返回栈并设置菜单；内容导航调用PushContent压栈。返回栈及菜单状态属于MainViewModel，未增加导航宿主或页面工厂。ViewLocator实现IDataTemplate，以命名替换、反射和Activator创建视图，按类型缓存并设置DataContext。

2.2 功能模块设计

菜单提供概览、书库、书摘、计划、统计及设置六个入口。书库进入详情或书籍编辑；详情进入书籍编辑、书摘编辑、计划编辑及记录弹窗；书摘和计划列表进入相应编辑页。内容返回保留上一级业务上下文，菜单切换清空历史栈。

导航顺序为定位目标单例、SetParameter、切换内容或压栈、Loaded公开异步命令加载。加载失败保留目标页面，呈现错误及重试入口。重复同一单例内容目标在SetParameter之前拦截，避免覆盖栈中原参数；不同书籍经返回书库再打开，不构造同类型详情的多份历史快照。

2.3 数据/数据库设计

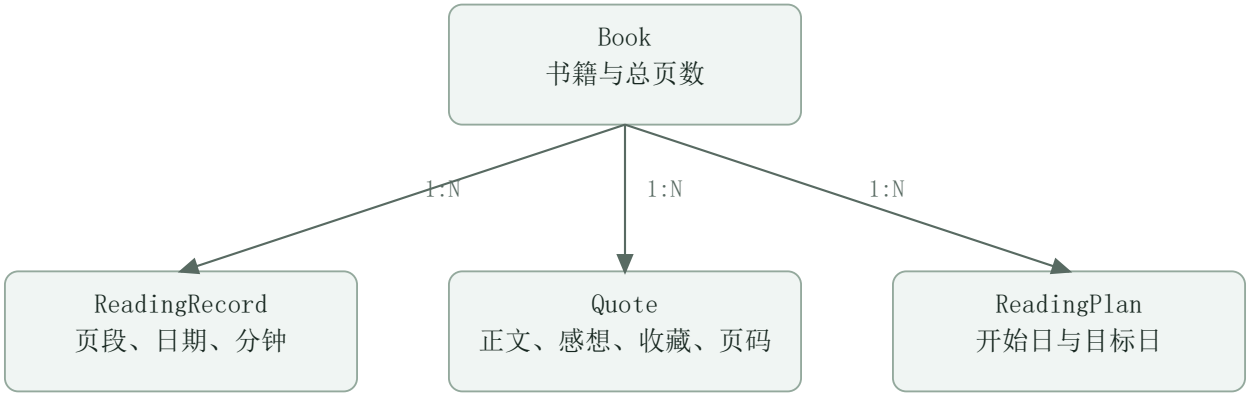


图 2 实际四表实体关系；子表BookId外键级联删除

Book含Id、Title、Author、Category、TotalPages、Description、CreatedAt和UpdatedAt。ReadingRecord含Id、BookId、ReadingDate、StartPage、EndPage和Minutes。Quote含Id、BookId、Text、Thoughts、可空Page、IsFavorite及创建/修改时间。ReadingPlan含Id、BookId、StartDate和TargetDate。数据库user_version为1，初始化事务创建表并启用外键。

记录约束为1 ≤ 起始页 ≤ 结束页 ≤ 总页数、分钟数为正整数、日期不晚于本地当天。当前页取最大结束页，无记录为0；累计页数对每条记录求闭区间长度之和，包含重读。完成日期为读至末页记录的最早日期。修改记录或总页数后由有效数据重新推导状态。

新建或编辑计划时，同一本书不能再关联第二个计划；普通新计划不能关联已完成书籍。旧备份中的多个完成计划可保留，但修改记录或总页数会使其同时变为未完成时，整个操作拒绝并提示先清理多余计划。未逾期的建议每日页数为剩余页数除以包含当天的剩余天数后向上取整；完成为0；逾期单列状态与剩余页数。删除计划不删除记录。

2.4 接口与类设计

最终采用与参考相同的语法树口径：只扫描UI与Library手写class/interface，包括默认访问级别；partial按工程及完整类型名去重；排除测试和生成代码，不计枚举、结构体或record。实际为66类、14接口、77个C#源文件、14个视图文件，其中业务视图10个。参考为63类、10接口、9视图；最低45类、7接口，实际数量已满足。

14项接口职责为数据库初始化、书籍存储、阅读记录存储、书摘存储、计划存储、偏好、阅读业务、统计、备份、提示、文件选择及三层导航。职责与实现逐一对应，未通过空接口或人为拆分补足数量。主要类还包括四类实体、进度/计划/统计投影、共享编辑基类、各页ViewModel、平台视图和定位器。新增计划一致性规则类在事务中复用业务状态校验，避免不同写入路径与备份规则分歧。

完整逐类型名称、路径、行号及源文件哈希见docs/application-type-inventory.json和同名Markdown。参考复核结果保留于docs/reference-baseline-cloud.json，可用docs/tools/TypeInventory重跑；原PowerShell脚本仍保留。

2.5 技术选型说明

技术	实际版本和用途
.NET / C#	net8.0，SDK 8.0.424固定，跨平台桌面业务。
Avalonia	11.0.10；Desktop、Fluent、Inter同版本；行为包11.0.10.9。
CommunityToolkit.Mvvm	8.2.2；通知属性、RelayCommand及AsyncRelayCommand。
sqlite-net-pcl	1.8.116；本地SQLite及事务。
Microsoft.Extensions. DependencyInjection	8.0.0；真实IoC容器和构造注入。
xUnit / Moq	2.4.2 / 4.20.70；测试SDK17.6.0，Runner2.4.5。
Avalonia.Headless	11.0.10，仅用于自动化中的真实应用资源和视图机制测试。

版本遵循参考项目，不以升级替代原有架构。历史Linux桌面验证在Xvfb中启动Avalonia应用；本次另在Windows真实桌面完成业务闭环。Headless断言与真实桌面操作分别记录。首次完整构建有3条NETSDK1206提示，源于固定SQLite原生依赖含旧Alpine RID；Linux x64与Windows x64实测正常，不据此宣称支持Alpine。

3 系统实现

3.1 关键模块的核心实现

以下摘录直接来自交付源码。定位器构造中的注册均使用AddSingleton；末尾构建真实容器，属性由容器解析。

```
public ServiceLocator()
{
    var serviceCollection = new ServiceCollection();
    serviceCollection.AddSingleton<DataPaths>();
    serviceCollection.AddSingleton<IDatabaseService, DatabaseService>();
    serviceCollection.AddSingleton<IRootNavigationService, RootNavigationService>();
    serviceCollection.AddSingleton<IMenuNavigationService, MenuNavigationService>();
    serviceCollection.AddSingleton<IContentNavigationService, ContentNavigationService>();
```

```
        serviceCollection.AddSingleton<MainWindowViewModel>();
        serviceCollection.AddSingleton<InitializationViewModel>();
        serviceCollection.AddSingleton<MainViewModel>();
        serviceCollection.AddSingleton<OverviewViewModel>();
        serviceCollection.AddSingleton<BookListViewModel>();
        serviceCollection.AddSingleton<BookDetailViewModel>();
        serviceCollection.AddSingleton<IReadingService, ReadingService>();
        serviceCollection.AddSingleton<IBookStorage, BookStorage>();

        serviceCollection.AddSingleton<ReadingPlanListViewModel>();
        serviceCollection.AddSingleton<ReadingPlanEditViewModel>();
        serviceCollection.AddSingleton<StatisticsViewModel>();
        serviceCollection.AddSingleton<SettingsViewModel>();
        _serviceProvider = serviceCollection.BuildServiceProvider();
    }
    public MainWindowViewModel MainWindowViewModel => _serviceProvider.GetService<MainWindowViewModel>()!;
    public InitializationViewModel InitializationViewModel => _serviceProvider.GetService<InitializationViewModel>()!;
    public MainViewModel MainViewModel => _serviceProvider.GetService<MainViewModel>()!;
    public OverviewViewModel OverviewViewModel => _serviceProvider.GetService<OverviewViewModel>()!;
```

内容导航保留常量switch映射与定位器调用。在阅记新增的忙碌页面和重复单例保护之后统一注入参数并压栈，未引入其他导航机制。

```
    public void NavigateTo(string view, object? parameter = null)
    {
        ViewModelBase target = view switch
        {
            ContentNavigationConstant.BookDetailView => ServiceLocator.Current.BookDetailViewModel,
            ContentNavigationConstant.BookEditView => ServiceLocator.Current.BookEditViewModel,
            ContentNavigationConstant.QuoteEditView => ServiceLocator.Current.QuoteEditViewModel,
            ContentNavigationConstant.ReadingPlanEditView => ServiceLocator.Current.ReadingPlanEditViewModel,
            _ => throw new ArgumentException("未知或尚未实现的内容目标。", nameof(view))
        };
        var main = ServiceLocator.Current.MainViewModel;
        // Direct Add/Edit commands must obey the same busy-page boundary as menu/back/close.
        // Check before SetParameter so a rejected navigation cannot erase a cached editor's draft.
        if (main.Content?.IsBusy == true || main.ContentStack.Contains(target)) return;
        target.SetParameter(parameter!);
        main.PushContent(target);
    }
}
```

阅读业务通过有效记录计算当前进度、累计活动和完成日期，重读不会叠加当前进度。

```
    }
    public BookProgress Calculate(Book book, IEnumerable<ReadingRecord> records)
    {
        var all = records.Where(x => x.BookId == book.Id).ToArray();
        return new BookProgress
        {
            Book = book,
            CurrentPage = all.Select(x => x.EndPage).DefaultIfEmpty(0).Max(),
            TotalReadPages = all.Sum(x => (long)x.EndPage - x.StartPage + 1),
            TotalMinutes = all.Sum(x => (long)x.Minutes),
            CompletedOn = all.Where(x => x.EndPage == book.TotalPages).Select(x => (DateTime?)x.ReadingDate.Date).Min()
        };
    }
}
```

备份由格式版本和四类集合组成，每个持久化字段要求显式存在。恢复先验证完整文档，在串行事务边界捕获原快照并成功写出自动备份，再删除并恢复业务表。自动备份失败不替换，写入失败事务回滚，成功后用户再进入页面会重新加载；主题在独立偏好文件中。

3.2 核心功能与UI截图

以下十图保留2026年10月20日Linux真实X11窗口的功能展示，显示的是用户主动载入的虚构演示资料；它们是历史平台截图，不替代本次Windows验收。随后补充当前Windows窗口截图。非全部内容都能在同一窗口高度内展示时，页面提供滚动；截图不拼接或补画隐藏内容。



图 3 阅读概览：全量指标、在读书籍和近期记录



图 4 我的书库：搜索、状态筛选、分页与详情入口



图 5 书籍详情：进度、阅读活动与计划关联



图 6 书籍编辑：独立业务表单



图 7 书摘列表：虚构摘录、感想和收藏入口



图 8 书摘编辑：正文、感想、页码和收藏



图 9 计划列表：派生状态与建议页数



图 10 计划编辑：实际关联书籍与目标日期



图 11 统计：191页、210分钟、完成1本及近七日趋势



图 12 设置与数据：主题、导出、恢复和演示入口

本次Windows本机界面另行新建虚构测试书：120页，阅读10～20页、25分钟，摘录第15页并制定当天到期计划。累计活动为11页、25分钟，当前进度20/120页，剩余100页；与此前Linux演示库的191页等数据属于不同测试集，不混用统计。

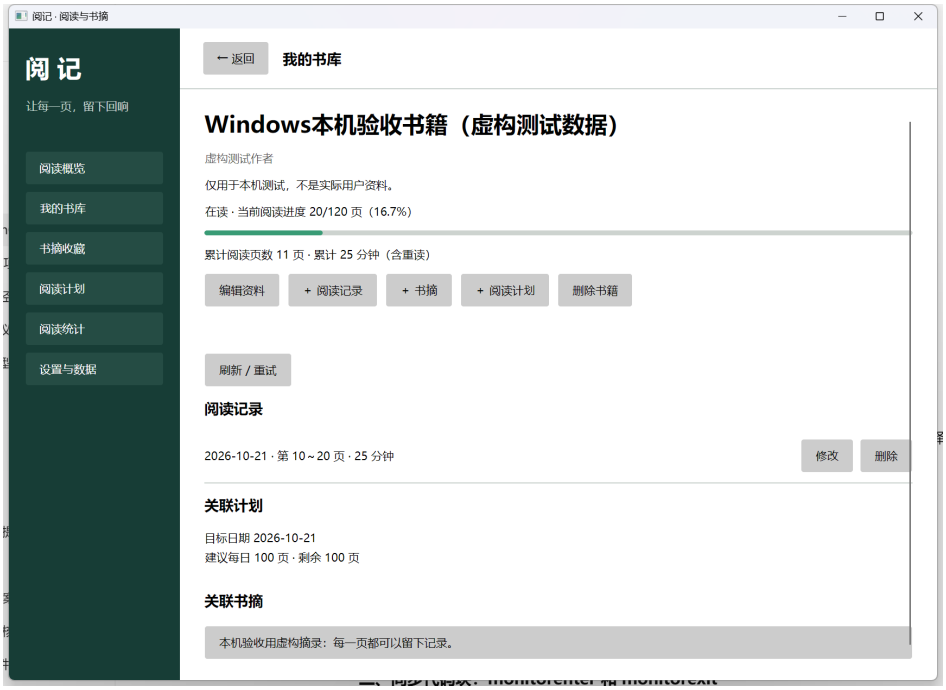


图 13 Windows本机：阅读记录、书摘与当天到期计划的关联详情

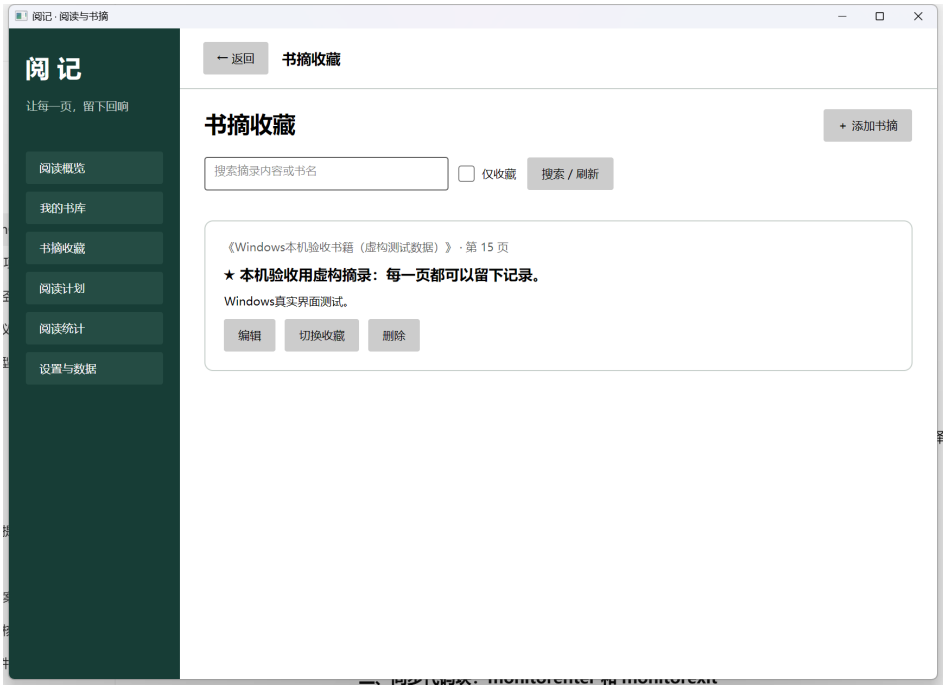


图 14 Windows本机：书摘收藏后显示星号，正文与感想保持



图 15 Windows本机：实际统计11页、25分钟、0本完成



图 16 Windows本机：删除后恢复并正常重启，业务数据与深色主题保留

3.3 难点和解决方案

实际问题	已执行处理及验证
参考源码最初不可用	Gitee代理CONNECT403，本地无参考目录；用户上传后同步副本，64文件哈希及语法树类型清单与原基线完全一致，才进入骨架开发。
命令绑定和目标错误	实际参考InitializationViewModel.cs:25命令为private，视图通过公开绑定访问；TodayViewModel.cs:76用内容服务传菜单目标。阅记公开命令、按层使用服务与常量，参考副本保持只读。
XAML与SQLite首轮失败	内部xmlns导致AXN0002，移至根节点；busy_timeout查询返回行，改用ExecuteScalar。修复后重跑当前阶段构建及测试。
单例异步旧数据	引入参数版本校验，旧请求不发布到新参数页面；加载命令发现版本变化继续加载新参数。延迟任务用例验证A不会覆盖B。
多次离开确认	主ViewModel共用防重入保护；返回、菜单、关闭不会并发打开多个确认窗口。保存失败保留草稿。
绑定诊断	事件命令显式绑定同一资源定位器中的单例，解决构造时DataContext未就绪；进度条使用静态模板，避免固定Fluent版本动画绑定。最终遍历十页绑定诊断为0。
原生选择器驱动	GTK路径自动补全曾干扰测试输入，应用正确提示文件不存在。通过真实列表选择同一文件后恢复成功，未用后台写库代替UI恢复。

本轮复核还发现加载与保存期间输入保护不足，以及计划状态变化与备份校验不一致。修复后，表单按字段合并未改值并保留用户修改，保存仅确认实际提交的快照；保存中新增修改保留在编辑页，新建成功回填主键避免重复插入。加载失败后可在编辑页重新加载。计划变更、记录修改与总页数修改在同一数据库事务中维护约束，拒绝不安全操作且不删除原数据。

后续修复还覆盖旧删除结果影响新导航、忙碌Loaded请求丢失、恢复期间导航，以及收藏与恢复交错。删除使用固定目标及参数版本检查；加载等待前次操作；所有页面忙碌时禁止菜单、返回、关闭和直接内容跳转。收藏由存储在同一事务内仅更新收藏位及时间，避免旧实体整体回写覆盖恢复数据。上述回归已纳入当前93项测试。

重复目标拦截、参数版本、草稿保护、串行写入和事务均为阅记新增业务正确性措施。参考README明确未解决线程与原子性，不能把这些新增设计称为参考项目原有机制。

4 系统测试

4.1 测试环境

2026年10月21日Windows本机实测使用Windows x64（系统版本10.0.26300）、.NET SDK 8.0.424、运行时8.0.30及Avalonia 11.0.10。业务测试使用隔离临时SQLite数据库，架构测试使用真实应用资源和Microsoft DI容器。真实桌面通过Windows UI Automation操作控件，并结合鼠标键盘与进程窗口截图；未用Headless运行冒充Windows桌面。

保留历史平台证据：2026年10月20日在Debian 13 / Linux x64、Xvfb 21.1.16 / 1440×1000环境完成原68项测试与桌面流程；后续收藏竞态修复后Linux同样完成93项测试。历史数字仅对应当时版本，当前自动化测试总数为93，不把多个平台的同一测试相加。

Windows工作区和v4源码包独立解包后，均执行锁定还原、Release构建和完整测试，结果各为93通过、0失败、0跳过。依赖还原使用本机现有NuGet配置与缓存，未隔离全部缓存；这不证明首次还原能够离线完成。两次构建均0错误，存在3条固定SQLite依赖旧Alpine RID的NETSDK1206提示。

```
dotnet restore ReadJournal.sln --locked-mode
dotnet build ReadJournal.sln -c Release --no-restore
dotnet test ReadJournal.sln -c Release --no-build --no-restore
```

正式配套源码为20261021-阅记-源码-v5.zip，共119个白名单文件、113557字节，低于10MB；无bin、obj、编译结果或运行数据库。SHA256为1fceab54b0774c4fe8b7a24bed70725356df59d59d0c88af88908088b475880b。本轮打包验证CRC正常、清单一致、每个文件与当前Code逐字节一致。

与实际独立测试的v4逐文件比较，v5仅README变化；其余118个文件（包括应用、测试和构建配置）完全一致。因此v5沿用相同代码的Windows93项验收证据，不声称修改README后又独立重跑了一遍v5测试。此前v4解包应用已使用另一个空数据目录实际启动，初始化版本1数据库并正常关闭；构建结果未回填ZIP。

另对当前Windows Release程序集运行独立架构探针：71项断言全部通过，覆盖14个VM单例、14个服务接口、13个反射视图、资源Current、真实容器、12个路由常量、参数、重复目标及返回栈。该71项与93项自动化测试分别统计，不将其写为164项业务单元测试。

Windows三个工作区启动日志和独立包启动日志均未输出绑定诊断。历史Linux曾验证演示数据、编辑重试和计划状态变化；本轮没有重新遍历所有历史UI边界，也未执行Alpine、大数据压力及所有CPU/系统组合。原生文件选择通过实际对话框完成，不宣称已实现全部步骤无人值守重放。

4.2 测试用例

下表按TRX中的测试类归组统计，每条数据行对应实际执行用例；总计93项，全部通过。

用例组与数量	主要实际覆盖	结果
架构9	真实容器、单例、构造注入、三层导航、反射缓存、失败重试、A/B切换及4个忙态直接路由场景。	通过
数据库3	初始化与持久化、外键事务回滚、初始化失败后重试。	通过
阅读业务12	闭区间页数、重读、当前进度、末页完成、非法输入、搜索分页及关联删除。	通过
编辑2	保存、放弃、取消离开及保存失败保留。	通过
扩展业务16	计划边界、统计、演示、无效备份、JSON往返、自动备份与恢复事务失败。	通过
导航时序2	延迟旧请求不覆盖新参数；离开确认不并发重复。	通过
编辑回归20	加载中草稿、首次加载门禁、失败重试、保存快照及新建ID回填。	通过
计划一致性8	重复关联、历史多个完成计划的状态变化、原子拒绝、安全编辑及备份往返。	通过
删除导航竞态6	固定删除目标、旧操作不误删新目标、不弹走新的页面或草稿。	通过
忙碌加载2	单例重新打开时等待旧操作并加载最新参数。	通过
设置导航4	数据操作期间菜单、返回与关闭保护，完成后恢复。	通过
概览架构3	最近记录查询由存储提供，VM职责及结果正确。	通过
恢复并发6	收藏/恢复次序、忙态菜单返回关闭、原子收藏与失败释放保护。	通过

对应原始证据为docs/evidence/local-windows-20261021/windows-full.trx和windows-clean.trx，工作区执行约4秒、独立解包约3秒；耗时仅为本机这次测量，不作为性能保证。运行时架构结果为architecture/runtime-checks.json。

Windows真实窗口另行完成：新库初始化；录入120页书籍；0起始页被拒绝；10～20页/25分钟保存；第15页书摘及计划录入；收藏切换；计划、统计和概览核对；原生JSON导出、恢复；确认删除书籍后四表各0；再次恢复后四表各1；切深色并正常关闭、重启。恢复后四表逐字段与恢复前快照一致，收藏保留，PRAGMA integrity_check为ok，foreign_key_check为空。删除后恢复产生的自动备份确为四表全空，证明备份记录了替换前状态。

4.3 测试截图

以下均为本次Windows实际进程窗口截图。93项通过的数量依据上节两份原始TRX及test.log，不使用旧68项终端截图冒充本次结果。

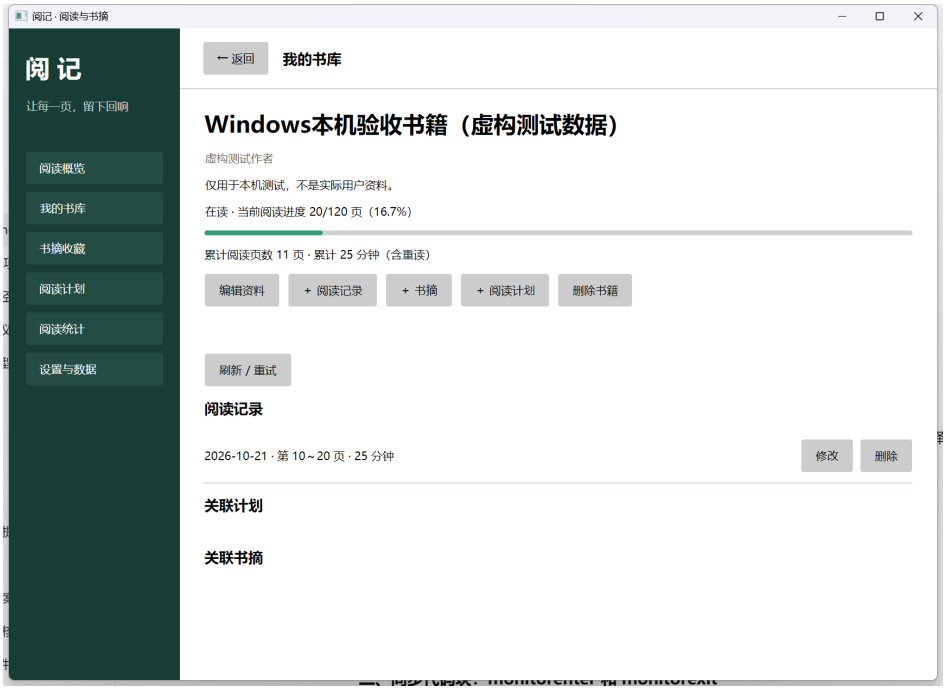


图 17 Windows真实阅读记录保存：10～20页按11页累计，进度20/120页

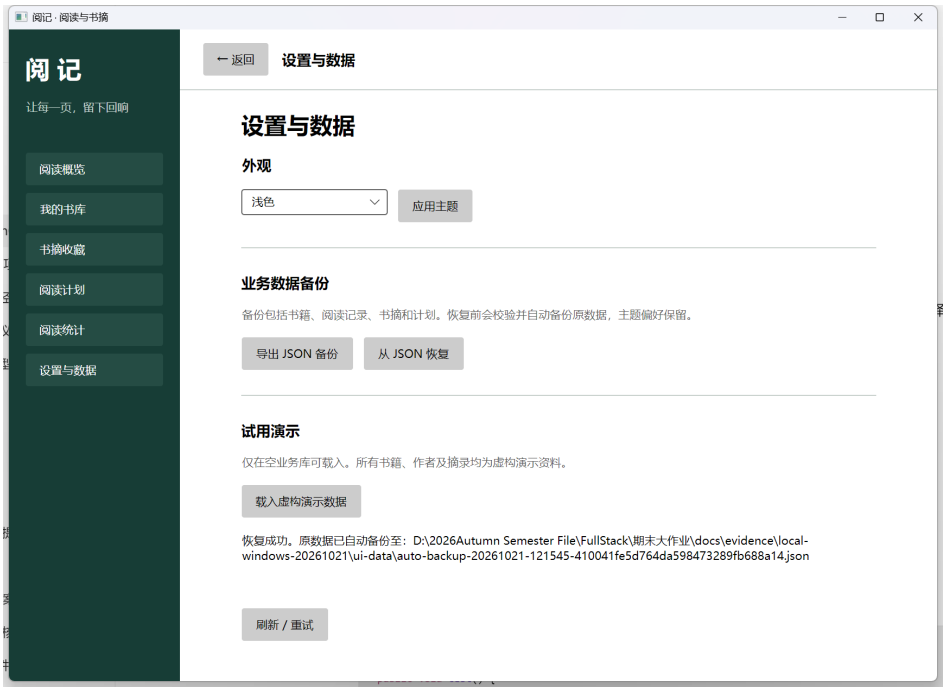


图 18 Windows真实备份恢复成功，并显示原数据自动备份路径

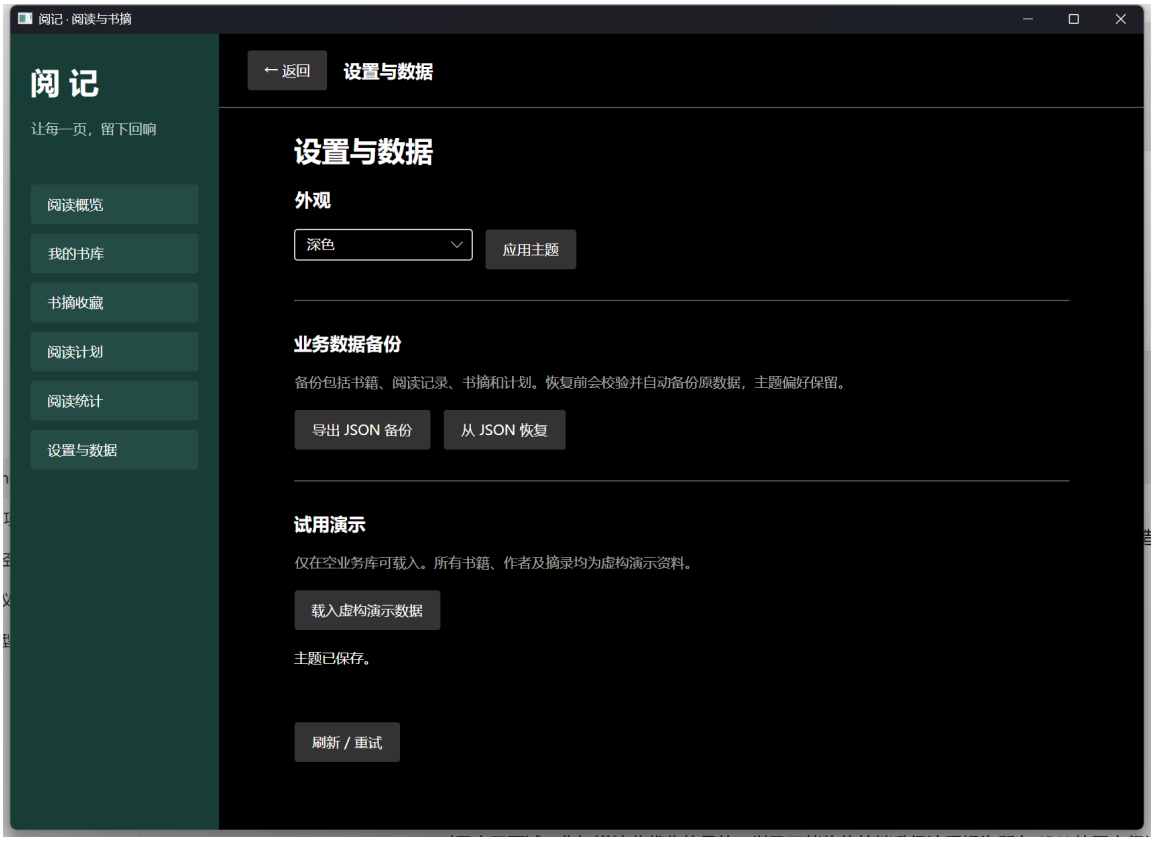


图 19 Windows实际应用深色主题，偏好保存成功



图 20 Windows正常关闭后重启：11页、25分钟及关联数据保留



图 21 v4独立解包后的实际首次启动，隔离空库初始化成功

截图对应业务数据来自明确标注的虚构测试书，不是实际用户阅读资料。测试摘要及逐字段核对结果存于validation.json、ui-final-data.json及ui-deletion-counts.json。本报告未对原始截图补画或拼接控件。

5 总结与展望

阅记完成十个业务页面及阅读管理闭环，保留参考项目真实IoC、单例定位器、三层导航、统一参数入口、主ViewModel返回栈和反射视图缓存。当前93项自动化测试、71项独立架构断言以及Windows本机真实桌面闭环通过；同时保留Linux历史验证。源码v5满足白名单、大小及逐文件一致性要求，应用与测试源码同已完成独立还原、构建、测试及启动的v4。

本修订版已同步Windows环境、测试结果、真实截图、异步竞态修复及源码包信息，配套源码v5；旧PDF及旧源码包保留作为历史版本。Windows通过结论来自本机实际验证，而非Linux推断。本机构建仍存在固定SQLite依赖的Alpine RID提示，未执行Alpine兼容、大数据压力或多设备同步测试。未来可在不破坏现有调用机制的前提下补充更多输入可用性与更大数据集测量；电子书阅读、联网检索和云同步属于第一版边界之外。

交付与验证项目	本版状态
规模与架构	66类、14接口、10业务页，八项机制对应；71项独立运行时架构断言通过。
应用与业务测试	Windows工作区及v4独立解包均93项通过；真实业务闭环和重启持久化通过。
数据安全与恢复	删除后四表全空；恢复后四表各1且逐字段一致；自动备份、完整性及外键核对正常。
源码与报告配对	本报告v2配套源码v5，119文件、113557字节；相比v4仅README修订，旧版保留。
继续改进的边界	后续可增加更大数据集测量、更多系统/CPU组合及输入可用性验证；目前不宣称全部场景穷尽通过。

本报告依据原始指导书模板、唯一开发规划、实际参考副本和最终实现编写。相关证据分别保留于来源记录、参考架构核对、开发执行记录、类型清单、测试结果和真实截图目录；参考源码及raw原始资料未改写。